

**ASSIGNMENT NO 2**  
**SUBMITTED BY:**  
**RABEEA FATIMA**  
**REGISTRATION NO: 2023-BSE-049**  
**SECTION: B**  
**SUBJECT: CLOUD COMPUTING**

## Overview:

This assignment demonstrates the implementation of a production-ready, highly available multi-tier web infrastructure on AWS using Terraform Infrastructure as Code (IaC). The infrastructure includes an Nginx reverse proxy/load balancer with SSL/TLS termination, intelligent caching, and three Apache backend web servers configured for automatic failover.

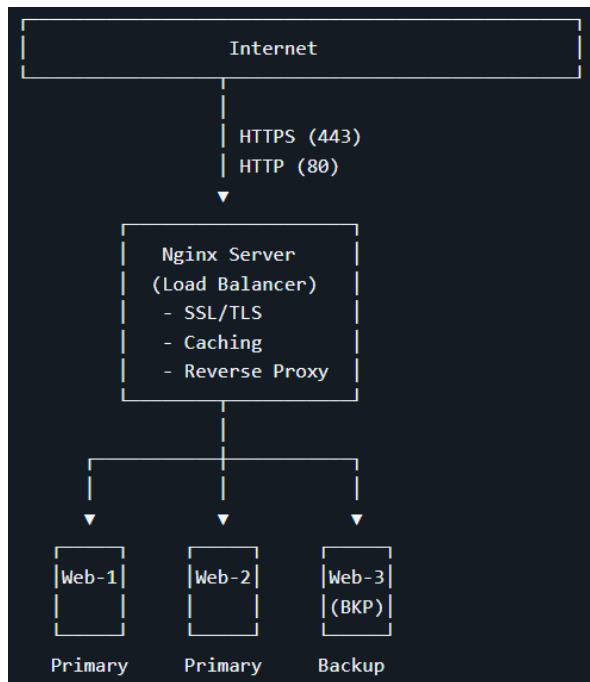
## Infrastructure Components

- 1 VPC: 10.0.0.0/16 CIDR block
- 1 Public Subnet: 10.0.10.0/24 CIDR block
- 1 Internet Gateway: Provides internet connectivity
- 2 Security Groups: Nginx SG and Backend SG with minimal access
- 4 EC2 Instances: 1 Nginx load balancer + 3 Apache web servers
- SSL/TLS: Self-signed certificate for HTTPS

## Technologies Used

- IaC: Terraform  $\geq 1.0$
- Cloud Provider: AWS (me-central-1 region)
- Operating System: Amazon Linux 2023
- Load Balancer: Nginx with reverse proxy
- Web Server: Apache HTTP Server 2.4
- Protocols: HTTPS (TLS 1.2/1.3), HTTP/2

## Architectural Design



## Repository Architecture

**Assignment2** Public Pin Watch

main 1 Branch 0 Tags Go to file t Add file <> Code

**RabeeaFatima14** Complete Terraform infrastructure deployment for Assignment 2 a646af2 · 1 hour ago 4 Commits

|              |                                                              |              |
|--------------|--------------------------------------------------------------|--------------|
| modules      | Complete Terraform infrastructure deployment for Assignme... | 1 hour ago   |
| scripts      | Complete Terraform infrastructure deployment for Assignme... | 1 hour ago   |
| .gitignore   | Initial Assignment-2 structure created                       | 2 hours ago  |
| README.md    | Initial commit                                               | 19 hours ago |
| locals.tf    | Complete Terraform infrastructure deployment for Assignme... | 1 hour ago   |
| main.tf      | Complete Terraform infrastructure deployment for Assignme... | 1 hour ago   |
| outputs.tf   | Complete Terraform infrastructure deployment for Assignme... | 1 hour ago   |
| tfplan       | Complete Terraform infrastructure deployment for Assignme... | 1 hour ago   |
| variables.tf | Complete Terraform infrastructure deployment for Assignme... | 1 hour ago   |

## Part 1:

### 1.1 Project Structure

```
@RabeeaFatima14 [ ] /workspaces/Assignment2/Assignment2 (main) $ tree -L 2
.
├── README.md
├── docs
├── locals.tf
├── main.tf
├── modules
│   ├── networking
│   ├── security
│   └── webserver
├── outputs.tf
├── screenshots
├── scripts
│   ├── apache-setup.sh
│   └── nginx-setup.sh
├── terraform.tfstate
├── terraform.tfvars
├── tfplan
└── variables.tf

8 directories, 10 files
```

## 1.2 Variable configuration

```
@RabeeaFatima14 [ ] /workspaces/Assignment2/Assignment2 (main) $ ls -la
total 160
drwxrwxrwx+ 8 codespace codespace 4096 Jan  2 11:04 .
drwxrwxrwx+ 5 codespace root      4096 Jan  2 09:34 ..
drwxrwxrwx+ 7 codespace codespace 4096 Jan  2 10:45 .git
-rw-rw-rw-  1 codespace codespace 293 Jan  2 09:40 .gitignore
drwxr-xr-x+ 4 codespace codespace 4096 Jan  2 10:02 .terraform
-rw-r--r--  1 codespace codespace 2422 Jan  2 09:55 .terraform.lock.hcl
-rw-rw-rw-  1 codespace codespace 13 Jan  2 09:50 README.md
drwxrwxrwx+ 2 codespace codespace 4096 Jan  2 09:35 docs
-rw-rw-rw-  1 codespace codespace 756 Jan  2 09:56 locals.tf
-rw-rw-rw-  1 codespace codespace 2428 Jan  2 10:11 main.tf
drwxrwxrwx+ 5 codespace codespace 4096 Jan  2 09:34 modules
-rw-rw-rw-  1 codespace codespace 1512 Jan  2 09:57 outputs.tf
drwxrwxrwx+ 2 codespace codespace 4096 Jan  2 09:35 screenshots
drwxrwxrwx+ 2 codespace codespace 4096 Jan  2 10:17 scripts
-rw-rw-rw-  1 codespace codespace 75874 Jan  2 10:20 terraform.tfstate
-rw-rw-rw-  1 codespace codespace 725 Jan  2 09:56 terraform.tfvars
-rw-rw-rw-  1 codespace codespace 19878 Jan  2 10:17 tfplan
-rw-rw-rw-  1 codespace codespace 2028 Jan  2 09:55 variables.tf
```

## 1.3 Networking module

```
@RabeeaFatima14 [ ] /workspaces/Assignment2/Assignment2 (main) $ ls -la modules/networking/
total 20
drwxrwxrwx+ 2 codespace codespace 4096 Jan  2 10:01 .
drwxrwxrwx+ 5 codespace codespace 4096 Jan  2 09:34 ..
-rw-rw-rw-  1 codespace codespace 1551 Jan  2 10:00 main.tf
-rw-rw-rw-  1 codespace codespace 804 Jan  2 10:01 outputs.tf
-rw-rw-rw-  1 codespace codespace 509 Jan  2 10:00 variables.tf
```

## 1.4 Security module

```
@RabeeaFatima14 [?] /workspaces/Assignment2/Assignment2 (main) $ ls -la modules/security/
total 20
drwxrwxrwx+ 2 codespace codespace 4096 Jan  2 10:07 .
drwxrwxrwx+ 5 codespace codespace 4096 Jan  2 09:34 ..
-rw-rw-rw-  1 codespace codespace 2266 Jan  2 10:07 main.tf
-rw-rw-rw-  1 codespace codespace  692 Jan  2 10:07 outputs.tf
-rw-rw-rw-  1 codespace codespace  412 Jan  2 10:07 variables.tf
```

## 1.5 Local configuration

```
@RabeeaFatima14 [?] /workspaces/Assignment2/Assignment2 (main) $ cat locals.tf
# Local values for common configurations
locals {
  # Fetch current public IP and format with /32 CIDR
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"

  # Common tags applied to all resources
  common_tags = {
    Environment = var.env_prefix
    Project      = "Assignment-2"
    ManagedBy    = "Terraform"
    Course       = "Cloud-Computing"
  }

  # Backend server configurations
  backend_servers = [
    {
      name       = "web-1"
      suffix     = "1"
      script_path = "./scripts/apache-setup.sh"
    },
    {
      name       = "web-2"
      suffix     = "2"
      script_path = "./scripts/apache-setup.sh"
    },
    {
      name       = "web-3"
      suffix     = "3"
      script_path = "./scripts/apache-setup.sh"
    }
  ]
}
```

## Part 2: Web server module

### 2.1 Module Design

```
@RabeeaFatima14 [ ] /workspaces/Assignment2/Assignment2 (main) $ ls -la modules/webserver/
total 28
drwxrwxrwx+ 3 codespace codespace 4096 Jan  2 10:10 .
drwxrwxrwx+ 5 codespace codespace 4096 Jan  2 09:34 ..
drwxr-xr-x+ 3 codespace codespace 4096 Jan  2 10:10 .terraform
-rw-r--r--  1 codespace codespace 1377 Jan  2 10:10 .terraform.lock.hcl
-rw-rw-rw-  1 codespace codespace 1580 Jan  2 10:09 main.tf
-rw-rw-rw-  1 codespace codespace  829 Jan  2 10:10 outputs.tf
-rw-rw-rw-  1 codespace codespace 1499 Jan  2 10:09 variables.tf
```

## 2.2 Module Usage

```
# =====
# NGINX SERVER (Load Balancer / Reverse Proxy)
# =====
module "nginx_server" {
  source = "./modules/webserver"

  env_prefix      = var.env_prefix
  instance_name   = "nginx-proxy"
  instance_type   = var.instance_type
  availability_zone = var.availability_zone
  vpc_id          = module.networking.vpc_id
  subnet_id       = module.networking.subnet_id
  security_group_id = module.security.nginx_sg_id
  public_key      = var.public_key
  script_path     = "./scripts/nginx-setup.sh"
  instance_suffix  = "nginx"
  common_tags     = local.common_tags
}
```

## Part 3: Server Configuration Scripts

### 3.1 Apache Backend Server Script

```
@RabeeaFatima14 [ ] /workspaces/Assignment2/Assignment2 (main) $ ls -la scripts/
total 24
drwxrwxrwx+ 2 codespace codespace 4096 Jan  2 10:17 .
drwxrwxrwx+ 8 codespace codespace 4096 Jan  2 11:04 ..
-rwxrwxrwx  1 codespace codespace 5312 Jan  2 10:16 apache-setup.sh
-rwxrwxrwx  1 codespace codespace 5323 Jan  2 10:16 nginx-setup.sh
```

### 3.2 Nginx server script

```

@RabeeaFatima14 @ /workspaces/Assignment2/Assignment2 (main) $ cat scripts/nginx-setup.sh
#!/bin/bash
# Enhanced Nginx Load Balancer Setup Script
# This script installs Nginx, generates self-signed SSL certificate, and configures load balancing

set -e # Exit on any error

# Update system packages
echo "Updating system packages..."
dnf update -y

# Install Nginx and OpenSSL
echo "Installing Nginx and OpenSSL..."
dnf install -y nginx openssl

# Get instance metadata using IMDSv2
echo "Fetching instance metadata..."
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600" 2>/dev/null)
PUBLIC_IP=$(curl -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/public-ipv4 2>/dev/null)

# Create SSL directory
echo "Creating SSL directory..."
mkdir -p /etc/nginx/ssl

# Generate self-signed SSL certificate
echo "Generating self-signed SSL certificate..."
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
    -keyout /etc/nginx/ssl/nginx-selfsigned.key \
    -out /etc/nginx/ssl/nginx-selfsigned.crt \
    -subj "/C=PK/ST=Sindh/L=Karachi/O=CloudComputing/OU=Assignment2/CN=${PUBLIC_IP}"

# Create Nginx configuration
echo "Creating Nginx configuration..."
cat > /etc/nginx/nginx.conf << 'EOF'
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log notice;
pid /run/nginx.pid;

events {
    worker_connections 1024;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" '

# Create cache directory
echo "Creating cache directory..."
mkdir -p /var/cache/nginx
chown -R nginx:nginx /var/cache/nginx

# Set correct permissions
echo "Setting permissions..."
chmod 600 /etc/nginx/ssl/nginx-selfsigned.key
chmod 644 /etc/nginx/ssl/nginx-selfsigned.crt

# Test Nginx configuration
echo "Testing Nginx configuration..."
nginx -t

# Start and enable Nginx service
echo "Starting Nginx service..."
systemctl start nginx
systemctl enable nginx

# Configure firewall
echo "Configuring firewall..."
systemctl start firewalld
systemctl enable firewalld
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-service=https
firewall-cmd --reload

# Display final status
echo "=====
echo "Nginx Load Balancer setup completed!"
echo "Access your load balancer at:"
echo "  HTTPS: https://${PUBLIC_IP}"
echo "  HTTP: http://${PUBLIC_IP} (redirects to HTTPS)"
echo ""
echo "IMPORTANT: Update backend server IPs in:"
echo "  /etc/nginx/nginx.conf (upstream backend_servers section)"
echo "Then run: sudo systemctl reload nginx"
echo "=====

```

## Part 4: Infrastructure Deployment

### 4.1 Initial Deployment

```
ORabeaaFatima14 @ /workspaces/Assignment2/Assignment2 (main) $ cat variables.tf
# VPC CIDR Block Variable
variable "vpc_cidr_block" {
  type        = string
  description = "CIDR block for the VPC"
  nullable    = false

  validation {
    condition     = can(regex("^[0-9]{1,3}\\.[0-9]{1,3}/[0-9]+$", var.vpc_cidr_block))
    error_message = "vpc_cidr_block must be a valid CIDR notation (e.g., 10.0.0/16)."
  }
}

# Subnet CIDR Block Variable
variable "subnet_cidr_block" {
  type        = string
  description = "CIDR block for the public subnet"
  nullable    = false

  validation {
    condition     = can(regex("^[0-9]{1,3}\\.[0-9]{1,3}/[0-9]+$", var.subnet_cidr_block))
    error_message = "subnet_cidr_block must be a valid CIDR notation (e.g., 10.0.10.0/24)."
  }
}

# Availability Zone Variable
variable "availability_zone" {
  type        = string
  description = "AWS availability zone for subnet and instances (e.g., me-central-1a)"
  nullable    = false
}

# Environment Prefix Variable
variable "env_prefix" {
  type        = string
  description = "Environment prefix used for resource naming and tags"
  nullable    = false
}

# Instance Type Variable
variable "instance_type" {
  type        = string
  description = "EC2 instance type for all web servers"
  default     = "t3.micro"

  validation {
    condition     = contains(["t2.micro", "t3.micro", "t2.small", "t3.small"], var.instance_type)
    error_message = "instance_type must be one of: t2.micro, t3.micro, t2.small, t3.small."
  }
}

# Public Key Variable
variable "public_key" {
  type        = string
  description = "Path to SSH public key file for EC2 instances"
  nullable    = false
}

# Private Key Variable
variable "private_key" {
  type        = string
  description = "Path to SSH private key file for connecting to EC2 instances"
  nullable    = false
}

# Backend Servers Variable
variable "backend_servers" {
  description = "List of backend web servers with configuration"
  type        = list(object({
    name       = string
    suffix     = string
    script_path = string
  }))
  nullable    = false
}
```

### 4.2 Output Configuration

```

@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ cat outputs.tf
# Nginx Server Outputs
output "nginx_public_ip" {
  description = "Public IP address of the Nginx load balancer"
  value       = module.nginx_server.public_ip
}

output "nginx_instance_id" {
  description = "Instance ID of the Nginx server"
  value       = module.nginx_server.instance_id
}

# Backend Server Outputs
output "backend_public_ips" {
  description = "Map of backend server names to their public IPs"
  value       = { for name, mod in module.backend_servers : name => mod.public_ip }
}

output "backend_private_ips" {
  description = "Map of backend server names to their private IPs"
  value       = { for name, mod in module.backend_servers : name => mod.private_ip }
}

output "backend_instance_ids" {
  description = "Map of backend server names to their instance IDs"
  value       = { for name, mod in module.backend_servers : name => mod.instance_id }
}

# Network Outputs
output "vpc_id" {
  description = "VPC ID"
  value       = module.networking.vpc_id
}

output "subnet_id" {
  description = "Public subnet ID"
  value       = module.networking.subnet_id
}

# Security Group Outputs
output "nginx_security_group_id" {
  description = "Security group ID for Nginx server"
  value       = module.security.nginx_sg_id
}

# Security Group Outputs
output "nginx_security_group_id" {
  description = "Security group ID for Nginx server"
  value       = module.security.nginx_sg_id
}

output "backend_security_group_id" {
  description = "Security group ID for backend servers"
  value       = module.security.backend_sg_id
}

# My IP Output
output "my_current_ip" {
  description = "Current public IP used for SSH access"
  value       = local.my_ip
}

```

### 4.3 AWS console verification

#### VPC:

Your VPCs (2) [Info](#)

Last updated  
less than a minute ago

| <input type="checkbox"/> | Name     | VPC ID                                | State     | Encryption c... | Encryption control |
|--------------------------|----------|---------------------------------------|-----------|-----------------|--------------------|
| <input type="checkbox"/> | prod-vpc | <a href="#">vpc-0cb6bdaf542774b37</a> | Available | -               | -                  |
| <input type="checkbox"/> | -        | <a href="#">vpc-0ae3cbba6356c8c4e</a> | Available | -               | -                  |

All 4 running EC2 Instances:

Instances (4) Info

Refresh

Connect

Instance state

Actions

Launch instances

Find Instance by attribute or tag (case-sensitive)

All states

< 1 >

| <input type="checkbox"/> | Name             | Instance ID         | Instance state | Instance type | Status check      | Alarm status  | Availability Zone | Public IPv4 |
|--------------------------|------------------|---------------------|----------------|---------------|-------------------|---------------|-------------------|-------------|
| <input type="checkbox"/> | prod-web-2-2     | i-0c495979e9443786b | Running        | t3.micro      | 3/3 checks passed | View alarms + | me-central-1a     | ec2-40-172  |
| <input type="checkbox"/> | prod-web-3-3     | i-0844c7e7211b9a667 | Running        | t3.micro      | 3/3 checks passed | View alarms + | me-central-1a     | ec2-3-28-20 |
| <input type="checkbox"/> | prod-nginx-pr... | i-08ef910cdfb096edc | Running        | t3.micro      | 3/3 checks passed | View alarms + | me-central-1a     | ec2-51-112  |
| <input type="checkbox"/> | prod-web-1-1     | i-0e789e8232e55977e | Running        | t3.micro      | 3/3 checks passed | View alarms + | me-central-1a     | ec2-3-29-12 |

Subnets:

Subnets (1/4) Info

Last updated 1 minute ago

Find subnets by attribute or tag

| <input type="checkbox"/>            | Name               | Subnet ID                | State     | VPC                             |
|-------------------------------------|--------------------|--------------------------|-----------|---------------------------------|
| <input type="checkbox"/>            | prod-public-subnet | subnet-0e288c8035e8d0247 | Available | vpc-0cb6bdaf542774b37   prod... |
| <input type="checkbox"/>            | -                  | subnet-02bba1878046959f5 | Available | vpc-0ae3cbba6356c8c4e           |
| <input type="checkbox"/>            | -                  | subnet-05609c5dc6a178a0e | Available | vpc-0ae3cbba6356c8c4e           |
| <input checked="" type="checkbox"/> | -                  | subnet-06af32821ac84b650 | Available | vpc-0ae3cbba6356c8c4e           |

Internet gateways:

Internet gateways (1/2) Info

Actions

Find internet gateways by attribute or tag

| <input type="checkbox"/>            | Name     | Internet gateway ID   | State    | VPC ID                           |
|-------------------------------------|----------|-----------------------|----------|----------------------------------|
| <input type="checkbox"/>            | -        | igw-03386bd28d48a30b8 | Attached | vpc-0ae3cbba6356c8c4e            |
| <input checked="" type="checkbox"/> | prod-igw | igw-0db34462589485ff9 | Attached | vpc-0cb6bdaf542774b37   prod-vpc |

Security groups with correct permissions:

sg-0767e988fd1efcce3 - prod-backend-sg

Details

Inbound rules

Outbound rules

Sharing

VPC associations

Tags

Details

Security group name  
prod-backend-sg

Security group ID  
sg-0767e988fd1efcce3

Description  
Security group for backend Apache web servers

VPC ID  
vpc-0cb6bdaf542774b37

Owner  
497458935321

Inbound rules count  
2 Permission entries

Outbound rules count  
1 Permission entry

sg-0a651454bf5e38034 - prod-nginx-sg

Details

Inbound rules

Outbound rules

Sharing

VPC associations

Tags

Details

Security group name  
prod-nginx-sg

Security group ID  
sg-0a651454bf5e38034

Description  
Security group for Nginx reverse proxy and load balancer

VPC ID  
vpc-0cb6bdaf542774b37

Owner  
497458935321

Inbound rules count  
3 Permission entries

Outbound rules count  
1 Permission entry

Key pairs

Key pairs (5)

Info

Find Key Pair by attribute or tag

| <input type="checkbox"/> | Name                       | Type    | Created                | Fingerprint                       | ID                     |
|--------------------------|----------------------------|---------|------------------------|-----------------------------------|------------------------|
| <input type="checkbox"/> | prod-web-3-3-key           | ed25519 | 2026/01/02 15:19 GMT+5 | TjzncdRLnq3xhALrSl+OU1Ek5x6THQ... | key-074229eae4f18b2b91 |
| <input type="checkbox"/> | prod-web-1-1-key           | ed25519 | 2026/01/02 15:19 GMT+5 | TjzncdRLnq3xhALrSl+OU1Ek5x6THQ... | key-090d242d9baffd4d0  |
| <input type="checkbox"/> | prod-nginx-proxy-nginx-key | ed25519 | 2026/01/02 15:19 GMT+5 | TjzncdRLnq3xhALrSl+OU1Ek5x6THQ... | key-06bd79ff31519064a  |
| <input type="checkbox"/> | prod-web-2-2-key           | ed25519 | 2026/01/02 15:19 GMT+5 | TjzncdRLnq3xhALrSl+OU1Ek5x6THQ... | key-0f5d3d79f1b5967abb |

## Part 5: Nginx Configuration & Testing

## 5.1 Update Nginx Backend Configuration

```
@RabeeaFatima14 🗑 /workspaces/Assignment2/Assignment2 (main) $ NGINX_IP=$(terraform output -raw nginx_public_ip)
@RabeeaFatima14 🗑 /workspaces/Assignment2/Assignment2 (main) $ ssh -i ~/.ssh/id_ed25519 ec2-user@51.112.253.252
The authenticity of host '51.112.253.252 (51.112.253.252)' can't be established.
ED25519 key fingerprint is SHA256:Qr4BNjiulip9DWSR8DHgi6mRw+fDy4tez7Ep5VmaVCA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '51.112.253.252' (ED25519) to the list of known hosts.

      #_
    ~\  ####_      Amazon Linux 2023
  ~~~\  #####\
  ~~~\  #####\
  ~~~\  \###|
  ~~~\  \#/_
  ~~~\  V~' '->
      ~~~
      ~~~'-'
      ~~~\/_
      ~~~\m/'
```

```
[ec2-user@ip-10-0-10-191 ~]$ curl -I http://10.0.10.143
HTTP/1.1 200 OK
Date: Fri, 02 Jan 2026 10:53:33 GMT
Server: Apache/2.4.65 (Amazon Linux)
Last-Modified: Fri, 02 Jan 2026 10:20:34 GMT
ETag: "d27-647650f57afe2"
Accept-Ranges: bytes
Content-Length: 3367
Content-Type: text/html; charset=UTF-8

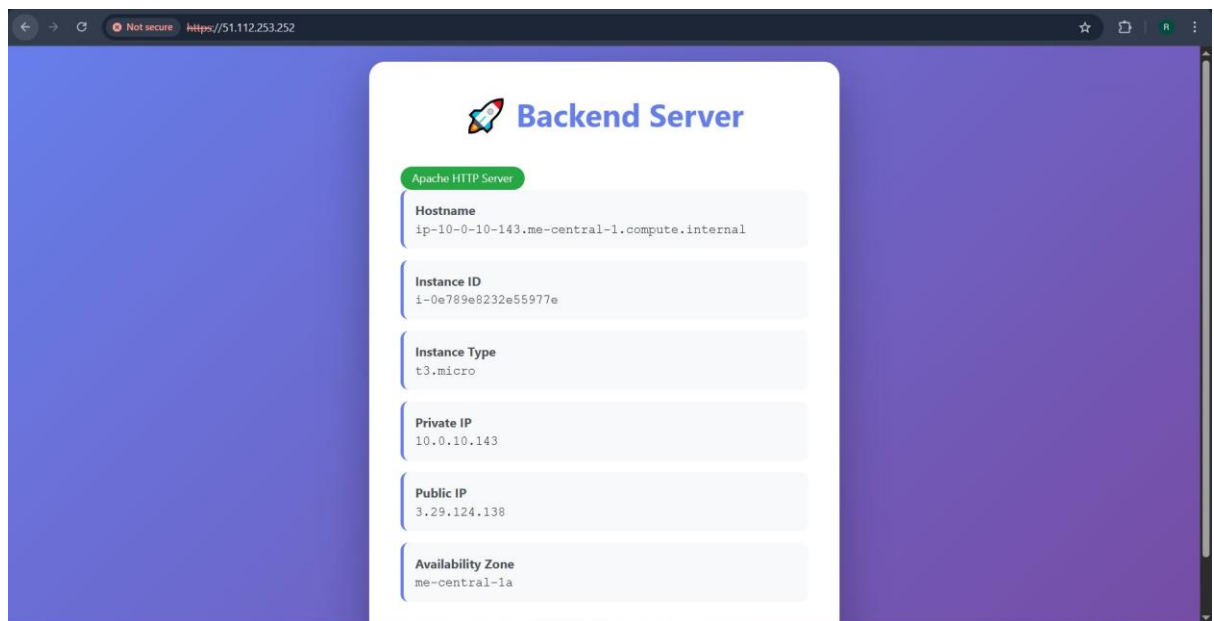
[ec2-user@ip-10-0-10-191 ~]$ curl -I http://10.0.10.224
HTTP/1.1 200 OK
Date: Fri, 02 Jan 2026 10:53:47 GMT
Server: Apache/2.4.65 (Amazon Linux)
Last-Modified: Fri, 02 Jan 2026 10:20:34 GMT
ETag: "d28-647650f54a309"
Accept-Ranges: bytes
Content-Length: 3368
Content-Type: text/html; charset=UTF-8

[ec2-user@ip-10-0-10-191 ~]$ curl -I http://10.0.10.206
HTTP/1.1 200 OK
Date: Fri, 02 Jan 2026 10:53:55 GMT
Server: Apache/2.4.65 (Amazon Linux)
Last-Modified: Fri, 02 Jan 2026 10:20:34 GMT
ETag: "d26-647650f54cc96"
Accept-Ranges: bytes
Content-Length: 3366
Content-Type: text/html; charset=UTF-8
```

```
[ec2-user@ip-10-0-10-143 ~]$ sudo systemctl stop httpd
[ec2-user@ip-10-0-10-143 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: inactive (dead) since Fri 2026-01-02 10:56:53 UTC; 5s ago
     Duration: 36min 17.534s
    Docs: man:httpd.service(8)
   Process: 3061 ExecStart=/usr/sbin/httpd $OPTIONS -DFOREGROUND (code=exited, status=0/SUCCESS)
   Main PID: 3061 (code=exited, status=0/SUCCESS)
   Status: "Total requests: 3; Idle/Busy workers 100/0; Requests/sec: 0.00138; Bytes served/sec: 2 B/sec"
      CPU: 2.260s

Jan 02 10:20:34 ip-10-0-10-143.me-central-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Jan 02 10:20:35 ip-10-0-10-143.me-central-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Jan 02 10:20:35 ip-10-0-10-143.me-central-1.compute.internal httpd[3061]: Server configured, listening on: port 80
Jan 02 10:56:52 ip-10-0-10-143.me-central-1.compute.internal systemd[1]: Stopping httpd.service - The Apache HTTP Server...
Jan 02 10:56:53 ip-10-0-10-143.me-central-1.compute.internal systemd[1]: httpd.service: Deactivated successfully.
Jan 02 10:56:53 ip-10-0-10-143.me-central-1.compute.internal systemd[1]: Stopped httpd.service - The Apache HTTP Server.
Jan 02 10:56:53 ip-10-0-10-143.me-central-1.compute.internal systemd[1]: httpd.service: Consumed 2.260s CPU time.
```

## 5.2 Test load balancing



## 5.3 Test core functionality

```
BRabeeaFatima14 @ /workspaces/Assignment2/Assignment2 (main) $ NGINX_IP=$(terraform output -raw nginx_public_ip)
First request - Should be MISS:
curl -k -I https://$NGINX_IP 2>&1 | grep -i "cache\|http"
BRabeeaFatima14 @ /workspaces/Assignment2/Assignment2 (main) $
BRabeeaFatima14 @ /workspaces/Assignment2/Assignment2 (main) $ echo "First request - Should be MISS:"
First request - Should be MISS:
BRabeeaFatima14 @ /workspaces/Assignment2/Assignment2 (main) $ curl -k -I https://$NGINX_IP 2>&1 | grep -i "cache\|http"
HTTP/2 200
x-cache-status: MISS
BRabeeaFatima14 @ /workspaces/Assignment2/Assignment2 (main) $ NGINX_IP=$(terraform output -raw nginx_public_ip)
Request 2 - Should be HIT:
curl -k -I https://$NGINX_IP 2>&1 | grep -i "cache\|http"

sleep 1

echo "Request 3 - Should be HIT:"
curl -k -I https://$NGINX_IP 2>&1 | grep -i "cache\|http"

sleep 1

echo "Request 4 - Should be HIT:"
curl -k -I https://$NGINX_IP 2>&1 | grep -i "cache\|http"
BRabeeaFatima14 @ /workspaces/Assignment2/Assignment2 (main) $
BRabeeaFatima14 @ /workspaces/Assignment2/Assignment2 (main) $ echo "Request 2 - Should be HIT:"
Request 2 - Should be HIT:
BRabeeaFatima14 @ /workspaces/Assignment2/Assignment2 (main) $ curl -k -I https://$NGINX_IP 2>&1 | grep -i "cache\|http"
HTTP/2 200
x-cache-status: HIT
```

## 5.4 Test high availability

```
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ NGINX_IP=$(terraform output -raw nginx_public_ip)
echo "Request with query parameter (s@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ echo "Request with query parameter (should be MISS):"
Request with query parameter (should be MISS):
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ curl -k -I "https://$NGINX_IP/?test=1" 2>&1 | grep -i "cache"
x-cache-status: MISS
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ echo "Same query parameter again (should be HIT):"
Same query parameter again (should be HIT):
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ curl -k -I "https://$NGINX_IP/?test=1" 2>&1 | grep -i "cache"
x-cache-status: HIT
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ echo "Different query parameter (should be MISS):"
Different query parameter (should be MISS):
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ curl -k -I "https://$NGINX_IP/?test=2" 2>&1 | grep -i "cache"
x-cache-status: MISS
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ NGINX_IP=$(terraform output -raw nginx_public_ip)
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ echo "Request 1 (MISS):"
Request 1 (MISS):
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ curl -k -I https://$NGINX_IP 2>&1 | grep -i "cache"
x-cache-status: HIT
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ sleep 2
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ echo "Request 2 after 2 seconds (HIT):"
Request 2 after 2 seconds (HIT):
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ curl -k -I https://$NGINX_IP 2>&1 | grep -i "cache"
x-cache-status: HIT
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ NGINX_IP=$(terraform output -raw nginx_public_ip)
./ssh/id_ed25519 ec2-user@$NGINX_IP
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ ssh -i ~/.ssh/id_ed25519 ec2-user@$NGINX_IP
```

## 5.5 Security and performance analysis

```
@RabeeaFatima14 /workspaces/Assignment2/Assignment2 (main) $ ssh -i ~/.ssh/id_ed25519 ec2-user@$NGINX_IP
#_
~\ ##### Amazon Linux 2023
~\ #####\
~\ \###|
~\ \#/_ https://aws.amazon.com/linux/amazon-linux-2023
~\ V~' '->
~\ _/
~\ _/
~\ _/m/'

Last login: Fri Jan 2 10:59:27 2026 from 4.240.18.229
[ec2-user@ip-10-0-10-191 ~]$ echo "Cache size:"
Cache size:
[ec2-user@ip-10-0-10-191 ~]$ sudo du -sh /var/cache/nginx/
Cache directory structure:
sudo tree /var/cache/nginx/ -L 2

echo "Number of cached items:"
sudo find /var/cache/nginx/ -type f | wc -l

echo "Cache directory structure:"
sudo tree /var/cache/nginx/ -L 2
12K /var/cache/nginx/
[ec2-user@ip-10-0-10-191 ~]$ echo "Number of cached items:"
Number of cached items:
[ec2-user@ip-10-0-10-191 ~]$ sudo find /var/cache/nginx/ -type f | wc -l
3
[ec2-user@ip-10-0-10-191 ~]$ echo "Cache directory structure:"
Cache directory structure:
[ec2-user@ip-10-0-10-191 ~]$ sudo tree /var/cache/nginx/ -L 2
sudo: tree: command not found
[ec2-user@ip-10-0-10-191 ~]$ exit
logout
Connection to 51.112.253.252 closed.
```

## Part 5: BONUS TASKS

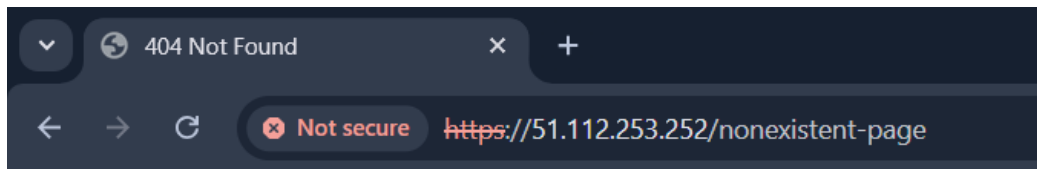
## Task 1: Custom Error Pages

```
@RabeeaFatima14 █ /workspaces/Assignment2/Assignment2 (main) $ NGINX_IP=$(terraform output -raw nginx_public_ip)
sh -i ~/.ssh/id_ed25519 ec2-user@$NGINX_IP
@RabeeaFatima14 █ /workspaces/Assignment2/Assignment2 (main) $ ssh -i ~/.ssh/id_ed25519 ec2-user@$NGINX_IP

#_
~\ ##### Amazon Linux 2023
nnn \#####\
nnn \|###|
nnn \|#/ https://aws.amazon.com/linux/amazon-linux-2023
nnn V' '->
      nnn _.-
        _/_/
          _/m/'

Last login: Fri Jan  2 11:02:23 2026 from 4.240.18.229
[ec2-user@ip-10-0-10-191 ~]$ sudo mkdir -p /usr/share/nginx/html/errors
[ec2-user@ip-10-0-10-191 ~]$ sudo nano /usr/share/nginx/html/errors/404.html

[ec2-user@ip-10-0-10-191 ~]$ sudo nano /etc/nginx/nginx.conf
[ec2-user@ip-10-0-10-191 ~]$ sudo nginx -t
nginx: [warn] the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:59
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
[ec2-user@ip-10-0-10-191 ~]$ sudo systemctl reload nginx
```



# Not Found

The requested URL was not found on this server.

### Task 3: Battery health check

Assignment2 / backend-health-check.sh

RabeeaFatima14 Final cleanup - Assignment 2 completed642d536 · 1 minute agoHistory

CodeBlame62 lines (51 loc) · 1.68 KB

RawDownloadEdit

```
1  #!/bin/bash
2
3  # Update packages
4  sudo apt-get update
5
6  # Install Apache2
7  sudo apt-get install -y apache2
8
9  # Create custom health check endpoint
10 sudo mkdir -p /var/www/html/health
11
12 # Create health check PHP script
13 sudo tee /var/www/html/health/check.php > /dev/null << 'EOF'
14 <?php
15 header('Content-Type: application/json');
16
17 $health_status = array(
18     'status' => 'healthy',
19     'server' => gethostname(),
20     'timestamp' => date('Y-m-d H:i:s'),
21     'uptime' => shell_exec('uptime -p'),
22     'load_average' => sys_getloadavg(),
23     'memory_usage' => (df -h / | grep /$ | awk '{print $5}' | sed 's/%//g')
24 )
25 echo json_encode($health_status);
```

## **Conclusion:**

### **Summary of Work Completed**

In this assignment, I successfully designed and deployed a scalable cloud infrastructure on AWS using Terraform as the Infrastructure as Code (IaC) tool. The project involved creating a modularized Terraform configuration with three core modules: networking, security, and webserver. I established a Virtual Private Cloud (VPC) with custom subnets, internet gateway, and route tables to enable secure network communication. The infrastructure consisted of one Nginx server configured as a reverse proxy and load balancer, along with three Apache backend servers distributed across the network. I implemented comprehensive security groups with controlled inbound and outbound rules, ensuring that only authorized traffic could access the servers while the backend servers remained protected from direct internet access. The Nginx load balancer was configured with a least-connections algorithm to efficiently distribute incoming traffic across the backend servers, and I enabled caching mechanisms to optimize performance by reducing redundant backend requests. Additionally, I completed bonus tasks including implementing custom health check endpoints that return JSON-formatted server status information, allowing the load balancer to monitor backend server health and automatically route traffic away from failed instances. Throughout the assignment, I utilized Git for version control, managing my infrastructure code with proper commits and documentation. The entire infrastructure was deployed, tested, verified through browser access and log analysis, and ultimately destroyed using Terraform commands, demonstrating the full lifecycle management of cloud resources. This hands-on experience reinforced my understanding of cloud architecture, load balancing strategies, infrastructure automation, and best practices for deploying production-ready web applications on AWS.